

Next.js Giriş

SSR/CSR, Routing, Layout ve Sayfa Yapısı

Session Akışı

Süre	Konu	Anahtar Kavramlar
0-20 dk	Next.js Nedir, Neden Var?	SSR vs CSR, framework vs library
20-50 dk	File-Based Routing	app/ klasörü, Link, dinamik route
50-70 dk	Layout ve Sayfa Organizasyonu	layout.tsx, nested layouts
70-90 dk	Pratik: Liste + Detay	dinamik route, params

1 🦋 Next.js Nedir, Neden Var?

🕒 0–20 dakika

1.1 React'tan Sonra Ne Eksik Kalıyor?

Şu ana kadar React ile arayüz geliştirmeyi öğrendik. Ama gerçek bir uygulamada React tek başına yeterli değil:

- **Routing** — birden fazla sayfa nasıl yönetilir?
- **SEO** — arama motorları sayfanızı nasıl indeksler?
- **İlk yükleme hızı** — boş HTML + büyük JS dosyası yavaş açılır
- **Sunucu tarafı veri çekme** — API key'leri client'ta göstermek istemeyiz
- **Image optimization, fonts, caching...**

Next.js Nedir?

Next.js, Vercel tarafından geliştirilen, **React üzerine kurulmuş bir framework**'tür. React'ın eksik bıraktığı kısımları (routing, SSR, optimizasyon, build sistemi...) hazır olarak sunar.

Library vs Framework: React bir kütüphanedir (siz çağırırsınız). Next.js bir framework'tür (o sizi çağırır — dosya yapısı, kuralları belli).

📌 Şirkette Neden Next.js?

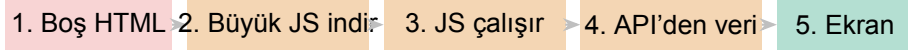
- Kurumsal projelerde **SEO** ve **performans** kritik
- File-based routing ile takım içinde tutarlı yapı
- Built-in optimizasyonlar (image, font, code-splitting)
- TypeScript, ESLint, CSS desteği out-of-the-box
- Vercel, AWS, kendi sunucumuz — her yere deploy edilebilir

1.2 Rendering Stratejileri — CSR vs SSR

1.2.1 CSR — Client-Side Rendering

Saf React (Vite, CRA) bu şekilde çalışır:

CSR (Client-Side Rendering):



Kullanıcı bu süre boyunca beyaz ekran görür

✓ Artılar:

Sayfa geçişleri hızlı
Sunucu yükü az
Basit deploy

✗ Eksiler:

İlk yükleme yavaş
SEO zayıf (bot boş HTML görür)
Yavaş cihazda JS yavaş çalışır

1.2.2 SSR — Server-Side Rendering

Next.js varsayılan olarak sayfaları sunucuda render eder:

SSR (Server-Side Rendering):

1. Sunucuda render 2. Hazır HTML gelir 3. İçerik görünür 4. JS yüklenir (hydrate)

Kullanıcı içeriği çok hızlı görür

✓ Artılar:

SEO mükemmel
İlk yükleme hızlı
Yavaş cihazda sorun yok

✗ Eksiler:

Sunucu gerekli
Her istek sunucuyu yorar
Biraz daha karmaşık

💡 Basit Bir Analoji

CSR: Restorana gider, masaya boş bir tabak konur, kendi yemeğinizi yaparsınız.

SSR: Yemek mutfakta hazırlanır, sıcak sıcak masaya gelir.

1.3 Karşılaştırma Tablosu

Özellik	CSR (saf React)	SSR (Next.js)
İlk yükleme	Yavaş — beyaz ekran	Hızlı — hazır HTML
SEO	Zayıf	Güçlü
Sayfa geçişleri	Hızlı	Hızlı (client-side nav)
Sunucu yükü	Düşük	Yüksek
Veri çekme	Browser'dan API'ye	Sunucudan API'ye
API key güvenliği	Sızabilir	Sunucuda kalır
Karmaşıklık	Düşük	Orta

⚠ Next.js Sadece SSR mı?

Hayır. Next.js, sayfa başına strateji seçmenize izin verir:

- **SSR** — her istekte sunucuda render
- **SSG** — build zamanında bir kere render (blog, landing page)
- **CSR** — belirli component'leri client'ta render ("use client")
- **ISR** — belirli aralıkla yeniden render

Şimdilik varsayılan davranışla (Server Components) çalışacağız.

2 📁 File-Based Routing

🕒 20–50 dakika

2.1 Proje Kurulumu

Next.js Projesi Oluşturma

```
npx create-next-app@latest my-app

# Sorulan seçenekler:
# TypeScript?    Yes
# ESLint?        Yes
# Tailwind CSS?  Yes (veya No --- bizim için opsiyonel)
# src/ dir?      Yes (önerilen)
# App Router?    Yes (modern yaklaşım)
# Turbopack?     Yes (hızlı)

cd my-app
npm run dev
# http://localhost:3000
```

2.2 App Router — Klasör = Route

File-Based Routing

Next.js'te **her klasör bir URL segment'idir** ve içindeki `page.tsx` dosyası o URL'in sayfasını oluşturur. Manuel route tanımları yoktur — dosya yapısı URL yapısıdır.

```
src/
├── app/
│   ├── layout.tsx → tüm sayfaların kabuğu
│   ├── page.tsx   → /
│   └── about/
│       ├── page.tsx → /about
│       └── blog/
│           ├── page.tsx → /blog
│           └── [slug]/
│               └── page.tsx → /blog/herhangi-bir-yazi
```

Dosya / Klasör	Anlamı
<code>page.tsx</code>	O route'un sayfası (UI)
<code>layout.tsx</code>	O route ve altındaki tüm sayfaların ortak çerçevesi
<code>loading.tsx</code>	Yüklenme sırasında gösterilecek UI
<code>error.tsx</code>	Hata olduğunda gösterilecek UI
<code>not-found.tsx</code>	404 sayfası
<code>[slug]</code>	Dinamik segment — URL'den parametre alır
<code>(group)</code>	Klasörü route'a katmaz, sadece organizasyon için

2.3 İlk Sayfa — page.tsx

src/app/page.tsx (anasayfa)

```
export default function HomePage() {
  return (
    <main>
      <h1>Anasayfa</h1>
      <p>Hosgeldin!</p>
    </main>
  );
}
```

src/app/about/page.tsx (/about)

```
export default function AboutPage() {
  return (
    <main>
      <h1>Hakkimizda</h1>
      <p>Biz kimiz, ne yaparız...</p>
    </main>
  );
}
```

💡 Önemli Nokta

Page component'i **default export** olmalıdır. İsmi ne olursa olsun (HomePage, Page, X) Next.js `export default`'u kullanır.

2.4 Sayfalar Arası Geçiş — Link Component

HTML'deki `<a href="">` kullanmak **tüm sayfayı yeniden yükler**. SPA hissini koruyabilmek için Next.js'in `<Link>` component'ini kullanırız.

Link Kullanımı

```
import Link from "next/link";

export default function Navbar() {
  return (
    <nav>
      <Link href="/">Anasayfa</Link>
      <Link href="/about">Hakkimizda</Link>
      <Link href="/blog">Blog</Link>
      <Link href="/blog/ilk-yazi">Ilk Yazı</Link>
    </nav>
  );
}
```

Tüm sayfa yeniden yüklenir
JS state sıfırlanır
Yavaş — sayfa flash eder
Hedef sayfa prefetch edilmez

<Link href="">

Sadece içerik değişir (client-side nav)
State korunur
Hızlı — akıcı geçiş
Hedef sayfa arka planda prefetch edilir

⚠ Dikkat

`<Link>` **sadece uygulama içi** linkler için kullanılır. Dış sayfalara (<https://google.com>) link verirken normal `<a>` kullanın.

2.5 Dinamik Route'lar — [slug] Klasörü

URL'in bir kısmının değişken olması gerektiğinde köşeli parantezli klasör kullanılır:

Klasör	Eşleşen URL'ler
blog/[slug]/page.tsx	/blog/hello, /blog/abc, /blog/123
products/[id]/page.tsx	/products/1, /products/42
users/[userId]/page.tsx	/users/ali, /users/zeynep

src/app/blog/[slug]/page.tsx

```
interface Props {
  params: Promise<{ slug: string }>;
}

export default async function BlogPostPage({ params }: Props) {
  {
    const { slug } = await params;

    return (
      <main>
        <h1>Yazi: {slug}</h1>
        <p>URL'den gelen slug: <strong>{slug}</strong></p>
      </main>
    );
  }
}
```

💡 Neden async ve await?

Next.js'in son sürümlerinde `params` bir Promise'dir. Component'i `async` yaparak `await params` ile değeri alırız. Server component'ler `async` olabilir — bu React'tan farklı bir özelliktir!

2.5.1 Çoklu Dinamik Segment

src/app/users/[userId]/posts/[postId]/page.tsx

```
interface Props {
  params: Promise<{ userId: string; postId: string }>;
}

export default async function UserPostPage({ params }: Props)
{
  const { userId, postId } = await params;
  // /users/ali/posts/3 --> userId="ali", postId="3"

  return (
    <main>
      <p>Kullanici: {userId}</p>
      <p>Post: {postId}</p>
    </main>
  );
}
```

2.6 Sayfada Veri Çekme

Server component'ler doğrudan `async` olabildiği için, veri çekme çok basitleşir:

Sunucu Tarafında Veri Çekme

```
// src/app/blog/page.tsx
interface Post {
  id: number;
  title: string;
  body: string;
}

export default async function BlogPage() {
  // Bu kod SUNUCUDA çalışır --- browser'a hic gelmez!
  const res = await fetch(
    "https://jsonplaceholder.typicode.com/posts?_limit=10"
  );
  const posts: Post[] = await res.json();

  return (
    <main>
      <h1>Blog</h1>
      <ul>
        {posts.map(p => (
          <li key={p.id}>{p.title}</li>
        ))}
      </ul>
    </main>
  );
}
```

🔧 Bu Ne Anlama Geliyor?

- useEffect + useState + loading yönetimi **yok**
- fetch doğrudan component içinde await ile
- Veri sunucuda çekilir, kullanıcıya hazır HTML olarak gelir
- Tarayıcı network sekmesinde fetch görmezsiniz!

3 Layout ve Sayfa Organizasyonu

 50–70 dakika

3.1 layout.tsx — Ortak Kabuk

Layout Nedir?

Layout, **birden fazla sayfanın paylaştığı UI çerçevesidir** — navbar, footer, sidebar gibi her sayfada görünecek parçalar. `layout.tsx` **aynı klasördeki** `page.tsx` ve **tüm alt klasörlerdeki** sayfaları sarar.

3.1.1 Root Layout (Zorunlu)

src/app/layout.tsx

```
import "../globals.css";
import Link from "next/link";

export const metadata = {
  title: "My App",
  description: "Next.js eğitimi",
};

export default function RootLayout({
  children,
}: {
  children: React.ReactNode;
}) {
  return (
    <html lang="tr">
      <body>
        <header style={{ padding: 16, background: "#2563eb", color: "white" }}>
          <nav style={{ display: "flex", gap: 16 }}>
            <Link href="/" style={{ color: "white" }}>Anasayfa</Link>
            <Link href="/blog" style={{ color: "white" }}>Blog</Link>
            <Link href="/about" style={{ color: "white" }}>Hakkımızda</Link>
          </nav>
        </header>

        <main style={{ padding: 24 }}>
          {children} { /* Sayfa içeriği buraya gelir */ }
        </main>

        <footer style={{ padding: 16, textAlign: "center", color: "#94a3b8" }}>
          <p>2026 Sirket Adi</p>
        </footer>
      </body>
    </html>
  );
}
```

⚠ Root Layout Zorunluluğu

src/app/layout.tsx **zorunludur** ve mutlaka <html> + <body> etiketlerini içermelidir. Diğer layout'lar opsiyoneldir ve bu etiketleri içermez.

3.2 Nested Layouts — İç İç Çerçeveler

Alt klasörlere de `layout.tsx` koyabiliriz. Bu, root layout'un **içinde** render olur. Örneğin sadece dashboard sayfalarında sidebar göstermek istiyorsak:

```
src/app/
├─ layout.tsx (root: header + footer)
├─ page.tsx
└─ dashboard/
    ├─ layout.tsx (dashboard: + sidebar)
    ├─ page.tsx
    └─ settings/
        └─ page.tsx
    └─ profile/
        └─ page.tsx
```

src/app/dashboard/layout.tsx

```
import Link from "next/link";

export default function DashboardLayout({
  children,
}: {
  children: React.ReactNode;
}) {
  return (
    <div style={{ display: "flex", gap: 24 }}>
      <aside style={{ width: 200 }}>
        <h3>Dashboard</h3>
        <nav style={{ display: "flex", flexDirection: "column",
          , gap: 8 }}>
          <Link href="/dashboard">Anasayfa</Link>
          <Link href="/dashboard/settings">Ayarlar</Link>
          <Link href="/dashboard/profile">Profil</Link>
        </nav>
      </aside>

      <section style={{ flex: 1 }}>
        {children}
      </section>
    </div>
  );
}
```

İç İç Render Sırası

/dashboard/settings URL'i şu şekilde render olur:

RootLayout (header + footer) → DashboardLayout (sidebar) → SettingsPage
Layout'lar iç içe geçer; her seviye {children} olarak bir sonrakini render eder.

3.3 Metadata — SEO için Önemli

Her sayfanın veya layout'un kendi metadata'sı olabilir. Bu, sayfa <head>'ine title, description gibi bilgiler ekler.

Statik Metadata

```
// src/app/about/page.tsx
import type { Metadata } from "next";

export const metadata: Metadata = {
  title: "Hakkimizda - My App",
  description: "Şirketimiz hakkında bilgiler",
};

export default function AboutPage() {
  return <h1>Hakkimizda</h1>;
}
```

Dinamik Metadata — URL'e Göre

```
// src/app/blog/[slug]/page.tsx
import type { Metadata } from "next";

interface Props {
  params: Promise<{ slug: string }>;
}

export async function generateMetadata({ params }: Props):
  Promise<Metadata> {
  const { slug } = await params;

  return {
    title: `${slug} - Blog`,
    description: `${slug} yazisi`,
  };
}
```

3.4 Özel Dosyalar — loading.tsx ve not-found.tsx

src/app/blog/loading.tsx

```
// Bu sayfanin verisi yuklenirken otomatik gosterilir
export default function Loading() {
  return (
    <div style={{ padding: 40, textAlign: "center" }}>
      <p>Blog yazilari yukleniyor...</p>
    </div>
  );
}
```

src/app/not-found.tsx

```
import Link from "next/link";

export default function NotFound() {
  return (
    <div style={{ padding: 40, textAlign: "center" }}>
      <h1>404 - Sayfa Bulunamadi</h1>
      <p>Aradiginiz sayfa mevcut degil.</p>
      <Link href="/">Anasayfaya don</Link>
    </div>
  );
}
```

💡 Özet — Özel Dosyalar

- page.tsx — sayfa içeriği
- layout.tsx — ortak kabuk
- loading.tsx — yüklenme göstergesi
- error.tsx — hata yakalama (Session 7’de detay)
- not-found.tsx — 404 sayfası

4 Pratik: Blog Listesi ve Detay Sayfası

 70–90 dakika

☰ Pratik Hedef

JSONPlaceholder API'sini kullanarak basit bir blog uygulaması yapalım:

1. / — Anasayfa
2. /blog — Tüm post'ların listesi (sunucu tarafında çekilir)
3. /blog/[id] — Bir post'un detayı (dinamik route)
4. Tüm sayfalar için root layout (navbar)
5. 404 sayfası

4.1 Dosya Yapısı

```
src/app/  
├─ layout.tsx  
├─ page.tsx  
├─ not-found.tsx  
└─ blog/  
    ├─ page.tsx  
    ├─ loading.tsx  
    └─ [id]/  
        └─ page.tsx
```

4.2 Adım 1 — Root Layout

src/app/layout.tsx

```
import Link from "next/link";
import "../globals.css";

export const metadata = {
  title: "Mini Blog",
  description: "Next.js ile blog uygulaması",
};

export default function RootLayout({
  children,
}: {
  children: React.ReactNode;
}) {
  return (
    <html lang="tr">
      <body>
        <header style={{ background: "#2563eb", padding: 16 }}>
          <nav style={{ display: "flex", gap: 16 }}>
            <Link href="/" style={{ color: "white" }}>Anasayfa
          </Link>
            <Link href="/blog" style={{ color: "white" }}>Blog
          </Link>
          </nav>
        </header>

        <main style={{ padding: 24, maxWidth: 800, margin: "0 auto" }}>
          {children}
        </main>
      </body>
    </html>
  );
}
```

4.3 Adım 2 — Anasayfa

src/app/page.tsx

```
import Link from "next/link";

export default function HomePage() {
  return (
    <div>
      <h1>Hosgeldin!</h1>
      <p>Bu Next.js ile yapilmis basit bir blog uygulamasidir
      .</p>
      <Link href="/blog">Yazilara goz at</Link>
    </div>
  );
}
```

4.4 Adım 3 — Blog Listesi

src/app/blog/page.tsx

```
import Link from "next/link";

interface Post {
  id: number;
  title: string;
  body: string;
}

export default async function BlogListPage() {
  // Sunucuda calisir --- veri hazir HTML olarak gelir
  const res = await fetch(
    "https://jsonplaceholder.typicode.com/posts?_limit=10"
  );
  const posts: Post[] = await res.json();

  return (
    <div>
      <h1>Blog Yazilari</h1>
      <ul style={{ listStyle: "none", padding: 0 }}>
        {posts.map(post => (
          <li
            key={post.id}
            style={{
              padding: 16,
              marginBottom: 8,
              background: "white",
              borderRadius: 8,
              boxShadow: "0 1px 3px rgba(0,0,0,0.08)",
            }}
          >
            <Link
              href={`/blog/${post.id}`}
              style={{ color: "#2563eb", fontWeight: 600 }}
            >
              {post.title}
            </Link>
            <p style={{ color: "#64748b", marginTop: 8 }}>
              {post.body.substring(0, 80)}...
            </p>
          </li>
        ))}
      </ul>
    </div>
  );
}
```

4.5 Adım 4 — Loading Sayfası

src/app/blog/loading.tsx

```
export default function Loading() {  
  return (  
    <div style={{ padding: 40, textAlign: "center", color: "  
      #64748b" }}>  
      <p>Yazılar yükleniyor...</p>  
    </div>  
  );  
}
```

4.6 Adım 5 — Detay Sayfası (Dinamik Route)

src/app/blog/[id]/page.tsx

```
import Link from "next/link";
import { notFound } from "next/navigation";

interface Post {
  id: number;
  title: string;
  body: string;
}

interface Props {
  params: Promise<{ id: string }>;
}

export default async function BlogDetailPage({ params }: Props) {
  const { id } = await params;

  const res = await fetch(
    `https://jsonplaceholder.typicode.com/posts/${id}`
  );

  if (!res.ok) {
    notFound(); // 404 sayfasini goster
  }

  const post: Post = await res.json();

  return (
    <article>
      <Link href="/blog" style={{ color: "#64748b" }}>
        &larr; Geri
      </Link>
      <h1 style={{ marginTop: 16 }}>{post.title}</h1>
      <p style={{ lineHeight: 1.6, marginTop: 16 }}>
        {post.body}
      </p>
    </article>
  );
}
```

4.7 Adım 6 — 404 Sayfası

src/app/not-found.tsx

```
import Link from "next/link";

export default function NotFound() {
  return (
    <div style={{ padding: 40, textAlign: "center" }}>
      <h1>404</h1>
      <p>Aradığınız sayfa bulunamadi.</p>
      <Link href="/" style={{ color: "#2563eb" }}>
        Anasayfaya don
      </Link>
    </div>
  );
}
```

💡 Bu Uygulamada Neler Yaptık?

- File-based routing ile 3 sayfa oluşturduk
- <Link> ile sayfalar arası geçiş
- Dinamik route ([id]) ile detay sayfası
- Server component'te async fetch
- loading.tsx ile yüklenme göstergesi
- not-found.tsx ile 404 sayfası
- notFound() ile programatik 404 yönlendirme

5 Özet ve Sonraki Session

5.1 Bu Session'da Öğrendiklerimiz

- ✓ Next.js neden var, neyi çözer
- ✓ Library vs Framework farkı
- ✓ CSR vs SSR — artıları, eksileri
- ✓ App Router ile file-based routing
- ✓ `page.tsx`, `layout.tsx` dosyaları
- ✓ `<Link>` ile client-side navigasyon
- ✓ Dinamik route'lar (`[slug]`, `[id]`)
- ✓ Server component'te `async/await` ile veri çekme
- ✓ Nested layouts ile organize yapı
- ✓ Metadata (statik ve dinamik)
- ✓ `loading.tsx` ve `not-found.tsx`
- ✓ Pratik: blog listesi + detay uygulaması

5.2 Sıkça Sorulan Sorular

SSS

S: useState ve useEffect Next.js'te çalışmıyor mu?

C: Çalışır ama sadece **client component**'lerde. Bir component'in en üstüne `"use client"` yazarak onu client component yapabilirsiniz. Bunu Session 7'de detaylı göreceğiz.

S: Neden <a> yerine <Link>?

C: `<a>` tüm sayfayı yeniden yükler, JS state'i sıfırlanır. `<Link>` sadece içeriği değiştirir, SPA hissini korur ve hedef sayfayı arka planda prefetch eder.

S: Her sayfa varsayılan olarak SSR mi?

C: Evet, App Router'da varsayılan olarak server component'lerdir (her istekte veya build'de sunucuda render). Client component istiyorsanız `"use client"` yazmanız gerekir.

S: params neden Promise oldu?

C: Next.js'in son sürümlerinde `async` olmayan API'ler `async`'e çevriliyor. `await params` kullanarak değeri alırsınız. Server component'ler `async` olabildiği için bu doğal bir kalıptır.

5.3 Sonraki Session — Session 7: Next.js Gerçek Kullanım

→ Session 7 (13 Mayıs Çarşamba)

Gerçek proje senaryolarına yaklaşıyoruz:

- **API entegrasyonu** ve loading state yönetimi
- **Error handling** ve UI feedback (`error.tsx`)
- **Client component'ler** — `"use client"` ne zaman?
- **Auth flow** mantığı ve protected route
- **Pratik**: Login + protected page

